



VISÃO COMPUTACIONAL

AULA 3



Prof. Leonardo Gomes



CONVERSA INICIAL

Nesta aula, vamos continuar a debater o processamento de imagens e alguns dos principais filtros para atenuar ruídos, realçar pontos de interesse e bordas.

Objetivos da aula

Ao final desta aula, esperamos atingir os seguintes objetivos que serão avaliados ao longo dos nossos estudos da forma indicada.

Tabela 1 – Objetivos

1 - Corretamente analisar e aplicar filtros que atenuam ruído
2 - Corretamente analisar e aplicar algoritmos de detecção de borda
3 - Aplicar as técnicas para atenuar ruídos e realce de bordas com o openCV em linguagem de programação Python



TEMA 1 – FILTROS LINEARES

O filtro, no âmbito do processamento de imagens, consiste em uma “transformação” feita pixel a pixel. Ela é conseguida por meio de uma pequena matriz denominada *kernel*, ou máscara, que percorre toda a imagem, analisando cada pixel e seus vizinhos, visando definir um valor novo para o pixel central. Esse processo é feito por meio de cálculos, geralmente com uma média ponderada ou outro tipo de cálculo, baseado nos números da máscara (os pesos) contra a região analisada (os valores), para assim encontrar o valor do pixel central da região analisada. Outro nome para essa técnica é “convolução”, bastante conhecida no campo de processamento de sinais. Dependendo dos valores dessa máscara, obtemos um realce de pixels com determinada característica ou uma suavização da imagem, um borramento que atenua ruídos. Esses filtros, por considerarem os pixels espacialmente próximos em seus cálculos, são chamados de filtros espaciais.

Nas figuras a seguir analisamos um exemplo de um filtro, aplicando sua respectiva máscara.

Figura 1 – Aplicação de filtro passa-alta em uma imagem. Imagem original e o valor de cada pixel

16	14	9	7	11	3
3	19	4	2	6	4
15	3	13	15	8	15
6	4	13	9	11	14
4	16	7	2	16	10
11	9	10	3	18	2



Figura 1.1 – Máscara ou *kernel* e o resultado

-1	-1	-1
-1	8	-1
-1	-1	-1

Figura 1.2 – Resultado da multiplicação e soma dos valores

		35			

Nas figuras anteriores, o valor 35 é o resultado da soma das multiplicações de cada valor na máscara pelo pixel equivalente, e o 13 acaba sendo substituído por 35. Se essa máscara for aplicada em todos os pixels da imagem, obtemos um valor alto, que realçará os pixels que possuem um valor diferente dos vizinhos, também chamados na literatura de *blobs*. Por conta desse efeito de realce, dizemos que se trata de um filtro passa-alta. Por consequência, filtros que suavizam os *blobs* são chamados de passa-baixa.

Na sequência, apresentaremos dois filtros passa-baixa, importantes dentro do processamento de imagens, que possuem comportamento linear da função que as representam. Essa linearidade gera uma suavização das diferenças da imagem – um borramento.



1.1 Filtro da média

O filtro da média é um filtro passa-baixa que calcula a média da vizinhança para atribuir o valor de cada pixel, sendo que todos os pixels da máscara têm o mesmo peso. No *opencv*, a função que executa esse filtro é chamada de *blur*. Ela possui por parâmetros a imagem e as dimensões da máscara. No código a seguir, vemos um exemplo com as imagens de resultado.

```
Importcv2
img = cv2.imread("acar0.png")
resultado = cv2.blur(img, (7,7))
cv2.imshow("Resposta", resultado)
cv2.waitKey(0)
```

Figuras 2 e 2.1 – Imagem original na esquerda e depois do filtro da média na direita



Fonte: elaborado com base em DIY13/Shutterstock

No código anterior, a máscara da média utilizada possui tamanho 7x7; quanto maior a máscara, mais suavizada/borrada ficará a imagem.

1.2 Filtro gaussiano

Similar ao da média, o filtro da gaussiana é um filtro passa-baixa que suaviza a imagem. No entanto, enquanto a média utiliza uma máscara que dá o mesmo peso para todos os pixels da máscara, a gaussiana possui pesos maiores mais próximo do centro.



Figura 3 – Máscaras de filtro gaussiano tamanho 3x3, 5x5, 7x7 com sigma = 1

								0	0	1	2	1	0	0
								0	3	13	22	13	3	0
			1	4	7	4	1	1	13	59	97	59	13	1
			4	16	26	16	4	2	22	97	159	97	22	2
1	2	1	7	26	41	26	7	1	13	59	97	59	13	1
2	4	2	4	16	26	16	4	0	3	13	22	13	3	0
1	2	1	1	4	7	4	1	0	0	1	2	1	0	0

No *opencv*, utilizamos a função *GaussianBlur*, que precisa da imagem, das dimensões da máscara e de um parâmetro denominado sigma (σ), que dita como será a distribuição desses pesos na máscara. Um valor de sigma elevado significa que os pesos mais distantes e mais centrais terão valores parecidos, gerando imagens mais suavizadas, enquanto um valor baixo significa que os pesos do centro da máscara serão maiores, gerando imagens menos suavizadas. O filtro é ideal para atenuar ruídos do tipo gaussiano. A seguir, um código de exemplo no *opencv* e a imagem de resultado.

```
import cv2
img = cv2.imread("acar0.png")
resultado = cv2.GaussianBlur(img, (7,7),5)
cv2.imshow("Resposta", resultado)
cv2.waitKey(0)
```

Figura 4 e 4.1 – Imagem original na esquerda e depois do filtro gaussiano na direita



Fonte: elaborado com base em DIY13/Shutterstock



No código anterior, foi adotado um valor de $\sigma = 5$ e uma máscara de tamanho 7x7.

TEMA 2 – FILTROS NÃO-LINEARES

Além dos filtros lineares, vistos no tema anterior, temos também filtros cujo comportamento das funções que lhes representam não é linear e, por isso, não geram o mesmo borramento.

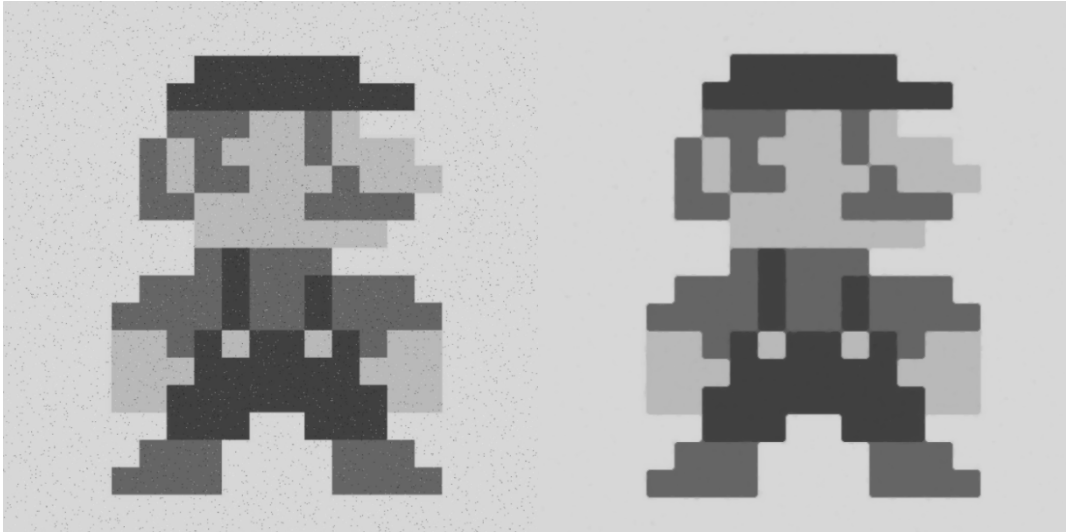
2.1 Filtro da mediana

O filtro da mediana funciona de maneira diferente dos filtros lineares, pois utiliza a mediana estatística dos pixels que estão na máscara como novo valor para o pixel alvo. Trata-se de um filtro interessante, pois não realiza uma suavização propriamente dita da imagem, porque o valor central do pixel será sempre um valor que existe dentro da imagem, preservando os contornos e bordas presentes nela. Este filtro funciona especialmente bem com ruído do tipo “sal e pimenta”.

No *opencv*, a função utilizada para esse filtro é a *medianBlur*, que utiliza dois parâmetros: a imagem e um valor inteiro positivo ímpar, que indique o tamanho da máscara quadrada que será utilizada.

```
import cv2
img = cv2.imread("marioSalPimenta.png")
resultado = cv2.medianBlur(img, 7)
cv2.imshow("Resposta", resultado)
cv2.waitKey(0)
```

Figura 5 e 5.1 – Imagem original na esquerda e depois do filtro da mediana na direita



Fonte: elaborado com base em ALTERNATIVE PUBLICIDAD/Shutterstock.

Observe que com o filtro da mediana todos os pontos do ruído “sal e pimenta” desapareceram e não se gerou borramento nos contornos da imagem como os filtros anteriores. No entanto, as quinas quadradas da imagem original ficaram levemente arredondadas.

2.2 Filtro bilateral

Por fim, apresentamos o filtro bilateral, este filtro funciona de forma semelhante ao filtro gaussiano, utilizando inclusive o mesmo princípio de média ponderada com quase as mesmas fórmulas, porém, além de considerar a distância espacial do pixel, também se é considerada a distância radial do pixel, ou seja, a informação do pixel em si, seja ela intensidade luminosa, cor ou profundidade. Pixels na máscara que sejam parecidos ao pixel alvo possuirão também um peso maior. Sendo assim, por considerar a distância radial e espacial, é chamado de filtro bilateral. As estratégias deste filtro visam preservar os contornos da imagem e diminuir o efeito de borramento.

No *opencv*, a função para utilizar filtro bilateral é a *bilateralFilter*. Como primeiro parâmetro utiliza-se a imagem, como segundo parâmetro se usa o tamanho da vizinhança espacial analisada. O segundo parâmetro é chamado de *sigmaColor*, que indica o peso dos pixels de cores próximas, e, por fim, o parâmetro *sigmaSpace*, que indica o peso que será dado para pixels espacialmente próximos. Em outras palavras, quanto maior o valor dos *sigmas*, maior será o borramento. A seguir, um código de exemplo e os resultados.



```
import cv2
img = cv2.imread("araraGaussiana.png")
resultado = cv2.bilateralFilter(img, 9, 75, 75)
cv2.imshow("Resposta", resultado)
cv2.waitKey(0)
```

Figura 6, 6.1 e 6.2 – A primeira, a imagem original com ruído gaussiano; a segunda, imagem após aplicação do filtro gaussiano com máscara de tamanho 7x7 e sigma 5; A terceira, após a aplicação do filtro bilateral com parâmetros 9,75,75





Fonte: elaborado com base em GORAN JAKUS/Shutterstock

Na imagem de resultado, vemos que o efeito de ruído estático foi significativamente atenuado pelos dois filtros. No entanto, o borramento nos contornos da imagem é muito mais perceptível no resultado do filtro gaussiano do que no filtro bilateral.

TEMA 3 – DETECÇÃO DE BORDAS

Além do tratamento de ruído, os filtros também são amplamente utilizados para detectar e realçar bordas na imagem. Entendemos por bordas as regiões de fronteira em que de um lado estão pixels predominantemente de uma cor e de outro lado, pixels predominantemente de outra. São os “contornos” ou “arestas” da imagem. Os filtros que serão apresentados a seguir são bastante suscetíveis a ruídos, e podem ser combinados com outros filtros de suavização já vistos.

3.1 Filtro de Sobel

O filtro de Sobel é utilizado para realçar contornos em imagens. Ele é composto por duas máscaras, uma para realçar linhas verticais e outra para realçar linhas horizontais. As máscaras de Sobel são apresentadas com configuração semelhante à figura a seguir.



Figura 7 – Máscaras do filtro de Sobel

-1	0	+1
-2	0	+2
-1	0	+1

+1	+2	+1
0	0	0
-1	-2	-1

Geralmente o filtro de Sobel é utilizado com a finalidade de detectar bordas mais do que realçá-las, pois gera valores muito intensos que descaracterizariam a imagem.

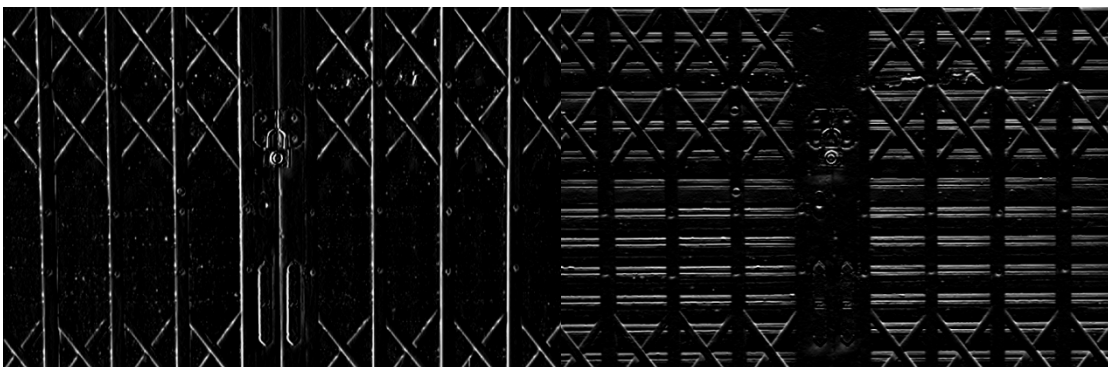
No código abaixo, vemos a implementação do filtro de Sobel no *opencv* e o resultado da execução. Ele é executado duas vezes com a função chamada *Sobel*, uma para bordas horizontais e outra para bordas verticais. Nos dois casos são usados 5 parâmetros; a imagem; o tipo dos dados do pixel, no caso, "CV_8U", significando inteiro e não negativo; o terceiro e quarto parâmetros indicam a direção do filtro: 1,0 significa na horizontal enquanto 0,1 significa na vertical; e por fim, *ksize*, que é o tamanho do *kernel* (máscara).

```
import cv2
img = cv2.imread("grade.png")
sobelH = cv2.Sobel(img, cv2.CV_8U, 1, 0, ksize = 3)
sobelV = cv2.Sobel(img, cv2.CV_8U, 0, 1, ksize = 3)
cv2.imshow("Horizontal", sobelH)
cv2.imshow("Vertical", sobelV)
cv2.waitKey(0)
```

Figura 8, 8.1 e 8.2 – Aplicação do filtro de Sobel: primeira imagem original, segunda e terceira imagens, resultado vertical e resultado horizontal



Créditos: OWEN1978/Shutterstock.



Fonte: elaborado com base em OWEN1978/Shutterstock.

Pelo resultado, vemos em branco as regiões de borda, vertical e horizontal, respectivamente.



3.1 Filtro de Laplace

O filtro de Laplace é muito popular para realçar bordas nas imagens, existem diversas variações de uso para esse filtro, porém a estratégia geral é detectar o pixel com uma intensidade luminosa muito diferente dos vizinhos. Na figura a seguir, vemos duas variações do filtro de Laplace.

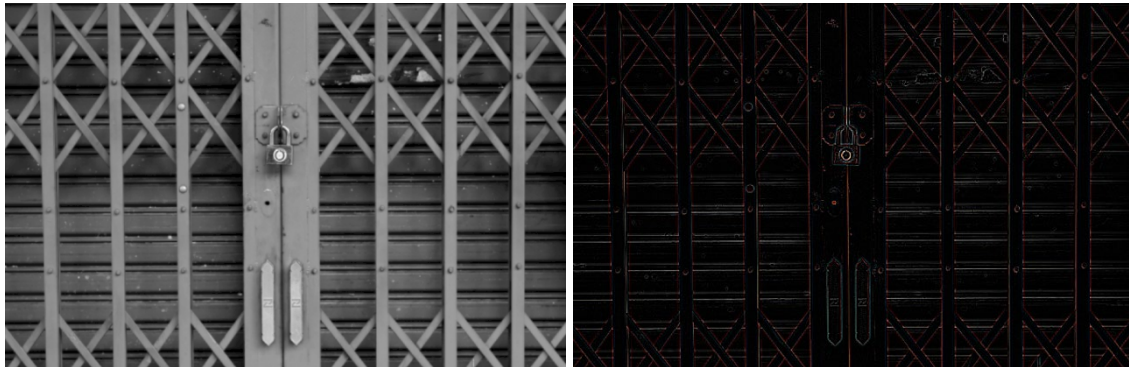
Figura 9 – Máscaras do filtro de Laplace

0	1	0	1	1	1
1	-4	1	1	-8	1
0	1	0	1	1	1

No código a seguir, vemos a demonstração do código de Laplace e imagem de exemplo do resultado no *opencv* através da função *Laplacian*. Ela adota dois parâmetros principais: a imagem e o tipo de dado do pixel, no caso, "CV_8U" significando inteiros não-negativos.

```
import cv2
img = cv2.imread("grade.png")
resultado = cv2.Laplacian(img, cv2.CV_8U)
cv2.imshow("Resposta", resultado)
cv2.waitKey(0)
```


Figuras 10 e 10.1 – Filtro de Laplace, primeira imagem original, segunda imagem, resultado do filtro



Fonte: elaborado com base em OWEN1978/Shutterstock.

O filtro de Laplace pode encontrar bordas nos dois sentidos da imagem e suas bordas são mais precisas e afinadas, o que permite técnicas de composição de imagem que destacam detalhes na imagem original. Esse assunto será debatido no próximo tema.

TEMA 4 – REALCE DE BORDAS

Com os filtros apresentados no tema anterior, Sobel e Laplace, é possível detectar bordas. No entanto, como já visto, a imagem gerada destaca somente as bordas e remove o fundo. Com a operação de subtração, já visto anteriormente, é possível combinar as duas imagens e realçar na imagem original as bordas que foram detectadas, tornando os detalhes da imagem ainda mais aparentes.

No código a seguir, esse processo é realizado através do *opencv*. Primeiro geramos a imagem com filtro de Laplace e na sequência ela é subtraída da imagem original. Além do código, serão apresentadas a seguir imagens de exemplo que ilustram esse processo.



```
import cv2
imgOrig = cv2.imread("moeda.png",0)
imgLaplace = cv2.Laplacian(imgOrig, cv2.CV_8U)
resultado = cv2.subtract(imgOrig, imgLaplace)
cv2.imshow("Laplace", imgLaplace)
cv2.imshow("Resultado", resultado)
cv2.waitKey(0)
```

Figuras 11, 11.1 e 11.2 – Na ordem, Imagem original; após filtro de Laplace; subtração da primeira pela segunda imagem



Créditos: OLEGRI/Shutterstock



Fonte: elaborado com base em OLEGRI/Shutterstock



Fonte: elaborado com base em OLEGRI/Shutterstock.

Chamamos de “regiões de alta-frequência” aqueles pixels com valor muito distinto de sua vizinhança e que geralmente carregam mais informações importantes da imagem. Uma técnica para detectá-los consiste em primeiro suavizar a imagem – por exemplo, com o filtro gaussiano – e na sequência subtrair a imagem original por essa imagem suavizada. Dessa forma, conseguimos detectar as regiões de alta frequência. Além disso, visando aumentar o grau de detalhes da imagem, podemos somar, na imagem original, essa imagem de alta frequência. Acompanhe o código e imagens de exemplo dessa estratégia na sequência.

```
import cv2
imgOrig = cv2.imread("raiox.png", 0)
imgGaussiana = cv2.GaussianBlur(imgOrig, (13, 13), 3)
imgRealce = 3 * cv2.subtract(imgOrig, imgGaussiana)
resultado = cv2.add(imgOrig, imgRealce)
cv2.imshow("Original", imgOrig)
cv2.imshow("Gaussiana", imgGaussiana)
cv2.imshow("Realce", imgRealce)
cv2.imshow("Resultado", resultado)
cv2.waitKey(0)
```


Figura 12, 12.1, 12.2 e 12.3 – Na ordem: Imagem original; suavização após filtro gaussiano; subtração da primeira pela segunda imagem; adição da primeira imagem com a terceira



Créditos: LENA SI/Shutterstock e adaptações.

No código anterior, a imagem de alta frequência foi multiplicada por 3 para justamente aumentar o grau de detalhes.

TEMA 5 – DETECTOR DE BORDAS DE CANNY

As aplicações para detecção de borda são as mais diversas: condução automatizada de veículos, classificação de imagens médicas, entre outras. E um dos métodos mais amplamente adotados para detecção de bordas é o algoritmo de Canny, desenvolvido nos anos 80 por John F. Canny. O algoritmo combina o filtro Gaussiano para reduzir ruídos e encontrar os pixels de alta

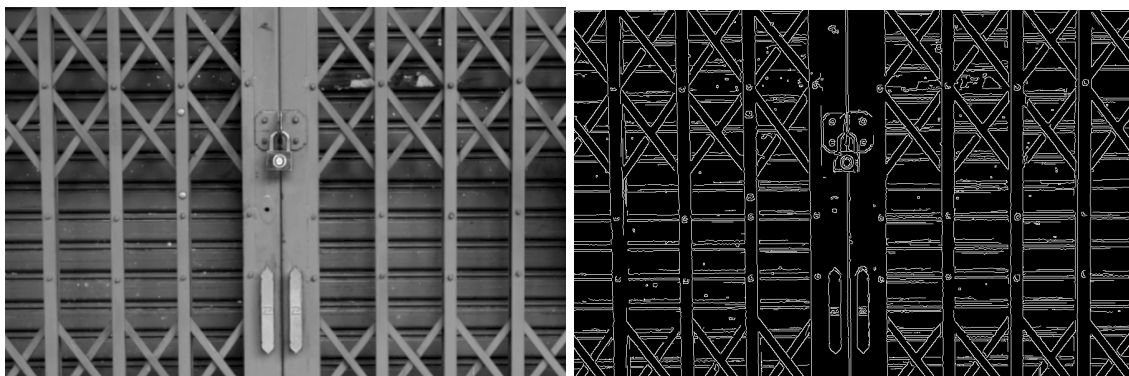


frequência, utiliza também o filtro de Sobel para detectar a direção para qual as bordas estão e com essas informações, combina os pixels de alta intensidade em linhas de borda com a espessura de um único pixel.

No *opencv*, o detector de bordas de Canny é executado através da função *Canny*, que recebe 3 parâmetros: o primeiro é a própria imagem; um limiar, que parametriza quando determinado pixel deve ser considerado uma borda e outro limiar, que parametriza o quanto as bordas devem agregar mais pixels. Em outras palavras, quanto menores os limiares parametrizados, mais bordas serão detectadas na imagem. Confira a seguir o código e imagens de exemplo.

```
import cv2
imgOrig = cv2.imread("grade.png",0)
resultado = cv2.Canny(imgOrig, 50, 150)
cv2.imshow("Resultado", resultado)
cv2.waitKey(0)
```

Figuras 13 e 13.1 – Na ordem, Imagem original; seguida da imagem resultado do detector de bordas de Canny



Fonte: Adaptado de OWEN1978/Shutterstock

Observe, na imagem final, as bordas detectadas possuem espessura de um único pixel. Se seguirmos por uma linha horizontal central na imagem poderíamos contabilizar quantos pixels brancos encontramos e utilizar esse valor para fazer uma estimativa de quantas barras compõem a grade, por exemplo.



FINALIZANDO

Nesta aula, discutimos os filtros aplicados nas imagens para destacar bordas ou suavizar a imagem e atenuar ruídos. Todas essas etapas são muito importantes dentro de um *pipeline* de visão computacional, visto que as imagens nem sempre estão prontas para serem trabalhadas da forma que desejamos. Com esse conhecimento, estamos mais aptos para detectar e realçar as regiões de interesse pertinentes para nossas aplicações.



REFERÊNCIAS

GONZALEZ, R. C.; WOODS, R. E. **Processamento Digital de Imagens**. 3ª Ed. São Paulo: Prentice Hall, 2010.

ARRUDA, E. P. **Fundamentos para o Desenvolvimento de Jogos Digitais**. Série Tekne. Bookman, 2014.

WINTERLE, P. **Geometria Analítica**. 2ª ed. São Paulo: Pearson, 2014.

FRIGERI, S. **Computação Gráfica**. 1ª ed. SAGAH, 2018

ANDALÓ, F. **Modelagem e Animação 2D e 3D para Jogos**. Érica, 2015.

SANTOS, F. FERREIRA, S. F. **Geometria Analítica**. ArtMed, 2009.

CHONG, A. **Animação Digital**. Coleção Animação Básica. AMGH, 2014.

BANIN, S. **Python 3 – Conceitos e Aplicações: uma abordagem didática**. 1ª ed. Érica, 2018.

BARELLI, F. **Introdução à visão computacional**. 1ª ed. Casa do Código, 2019.